# Predictive Modeling of Financial Time Series:
# Evaluating Non-Linear Classifiers for Market State Identification

Raunak Amanna

DADS7275: Machine Learning and Data Analytics

Chinthaka Pathum "Dinesh" Herath Gedara

4/19/2026

**Abstract**

Financial markets cycle between periods of calm growth and periods of extreme fear and volatility. Identifying these regime shifts one day in advance is one of the most practically valuable problems in quantitative risk management. This project frames market regime forecasting as a supervised binary classification problem, using 25 years of daily S&P 500 (SPY) and CBOE Volatility Index (VIX) data sourced through the Yahoo Finance API. A day is labeled High Risk (1) when the VIX closes above 20.0, and Stable (0) otherwise. To prevent look-ahead bias — a critical methodological safeguard — the target variable is shifted by one day so that today's market indicators predict tomorrow's regime. A strictly chronological train/test split (training on 2000–2019, testing on 2020–2025) is enforced to prevent temporal data leakage.

## 1B. Introduction and Problem Statement

I chose four models in increasing complexity; the first is the logistic regression model, which was the simple and baseline linear model. Next, I chose the Random Forest model as the baseline non-linear model because it is an ensemble method that uses decision trees to split the data and build criteria. I also selected an SVM with a RBF kernel to classify things well, and lastly, I used the LSTM model since they are capable of learning from sequential and temporal patterns. After developing all the models, I first evaluated them to compare the results of the linear and non-linear machine-learning models. I used precision, recall, F1 score, PR-AUC, and ROC-AUC to compare but recall was my main criterion because the cost difference for the error is extremely

asymmetrical especially for the high-risk class which has heavy penalties if wrong. From the evaluation, I noticed that the non-linear models did significantly better than the linear models, proving my hypothesis that market regimes can't be captured via linear boundaries, as market and real-life things are rarely linear. The best-performing model had a class 1 recall of [0.7219] which hit the project goal of exceeding 60% for this recall. From what I've learned, financial markets cycle around phases of good growth and fear and bad volatility so figuring these out in advance is extremely valuable, so a portfolio that can rotate into safer assets ahead of a high-risk regime has a plausible risk-management advantage over a static allocation. Quantifying that advantage would require a full backtest with transaction costs, which is outside the scope of this project — the results reported here are classification metrics, not return metrics.

Instead of trying to predict the future price of the S&P 500, which is extremely difficult, this project focuses on predicting the regime of the market–whether it's going to be a High Risk environment or a Stable one. This approach is much more feasible and a decision can easily be made regarding it. With that being said, this project seeks to build and compare multiple machine learning models. These models would be increasing in complexity with each iteration. It is to determine which model is best suited in intercepting the dangerous market conditions that we are trying to identify. It is important to note that when selecting the models, the recall was prioritized rather than accuracy. This is because a missed market crash could have disastrous effects.

# 2. Dataset and Methodology

## 2A. Dataset Description

All of the data points are sourced via the Yahoo Finance API using the yfinance Python library. I plan to download two assets with the time-period of Jan 1 2000 to Jan 1 2025. The first asset is SPY (an ETF for the S&P 500), SPY is used as the source of predictive features, and only the daily closing price is being downloaded as it is the only one of the four prices for a day that is most widely cited and meaningful. I will use the parameter auto_adjust=True to ensure that the historical prices are adjusted for dividends and stock splits to prevent artificial discontinuities. The second asset is the CBOE Volatility Index (^VIX), which will be used to provide the ground-truth labels. The daily closing value of the VIX is downloaded since that number represents the market's expected 30-day volatility, and it spikes sharply during periods of panic amongst investors.

The next preparation step is to merge both series based on their shared trading dates and to drop any remaining rows with missing values since even a single one could ruin the prediction model. After this, the dataset will contain approximately 6,290 rows. Since I plan on performing rolling calculations for feature engineering purposes later on, I will note that the first N rows of each derived column created in that process will be NaN, hence I will remove those later on. For

example: the 200-day moving average can't be computed until 200 days of data exist. With all this removal, a fully complete dataset will be left, that will begin around late 2000.

The target variable, Target_Next_Day, labels a day as High Risk (1) when VIX closes above 20.0 and Stable (0) otherwise. The single most important methodological safeguard is the one-day forward shift applied to this label: df['Target_Next_Day'] = df['Regime_Current'].shift(-1). The one-day shift is very vital as without it, the model would be using today's labels to predict today's labels! It wouldn't be using any features. This makes the forward shift essential rather than optional. The forward shift basically ensures that today's data is used to predict tomorrow's labels. The resulting classes are also reflective of market behavior. After the forward shift and removal of rows with incomplete rolling windows, 6,089 trading days remain, of which 62.2% are labeled Stable and 37.8% High Risk — a ratio of roughly 1.65 to 1. This is a moderate imbalance rather than a severe one, and it is handled through balanced class weighting within each model rather than through resampling, which preserves the underlying feature distributions. The point of the model is to predict the regime of the VIX, not directly the VIX, by designating everything under 20 as stable (0) and everything above 20 as risky (1).

---

## 2B. Methodology

**Feature Engineering**

Raw closing prices can't be directly used in this model since it's non stationary as SPY was traded at $100 in 2000 and $500 in 2024. Hence, I formed a set of scale independent features that are stationary, starting with RSI. RSI is a momentum oscillator and was computed with a period of 14 days. It ranges from 0 to 100 and is given by the formula RSI = 100- (100 / (1 + RS)), where RS is the ratio of average gains to average losses. For Daily Returns, they convert the non-stationary raw closing prices into a stationary form by calculating the percentage change in price from the previous day to today. The 50-day and 200-day simple moving averages are computed from the adjusted closing price, then converted from raw levels into stationary values. However, both of them were further modified to scale down the dependency on the raw closing price by using the formula (Price - SMA) / SMA and they lead to a value meaning the percentage deviation from the SMA. For example, −0.10 means it fell below the SMA by 10% regardless of the SMA being 120 or 480. Kept both SMAs as I needed both short term momentum (captured by 50-day) and long-term structure (captured by 200-day).

**Train/Test Split and Leakage Prevention**

The dataset is split into a training set (January 2000 – December 2019, 4,832 observations) and a test set (January 2020 – January 2025, 1,257 observations). This split is strictly chronological; a random split would let future data flow into the training set, producing predictions with no practical significance. The chronological ordering mimics how it is used in practice: predictions on the test set are based only on historical data. In our case, I fitted the StandardScaler only on

the training data and used the same one on the test data without refitting, avoiding any leakage of temporal trends.

**Feature Scaling**

A StandardScaler changes every feature so it has a mean of zero and a standard deviation of one. Here's the formula it uses: $z = (x - \mu) / \sigma$. It is really important to do, because certain our Machine Learning models require them, such as the Support Vector Machine(SVM) model and the LSTM model. We use the Standard Scaler for all our models to make things stable.

**Model Selection and Justification**

Four models are trained in order of increasing complexity. Each is evaluated against the previous one to isolate the contribution of additional complexity.

**Model 1 — Logistic Regression (Linear Baseline):** Logistic Regression computes a weighted linear combination of features, passes it through a sigmoid function to produce a probability, and applies a decision threshold. Its decision boundary is a hyperplane — it can only draw a straight line between classes. It is trained first because it is the simplest possible classifier. The EDA correlation analysis, which showed near-zero linear feature-target relationships, predicts that it will underperform. This prediction is verified empirically, providing data-driven justification for proceeding to non-linear models.

**Model 2 — Random Forest (Non-Linear Ensemble Benchmark):** Random Forest combines 100 decision trees, each trained on a bootstrap-sampled subset of the data and a random subset of features at each split. The ensemble majority vote approximates complex non-linear boundaries through the aggregation of simple conditional threshold rules. It can learn that low RSI combined with negative returns and negative SMA divergence simultaneously signals High Risk — a three-way interaction that no linear model can represent. Any recall improvement over the Logistic Regression directly quantifies the value of non-linear modeling for this problem.

**Model 3 — Support Vector Machine with RBF Kernel:** The SVM finds the decision boundary that maximizes the margin between classes. The RBF kernel $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ implicitly projects the four-dimensional feature space into a higher-dimensional representation where the classes become more linearly separable, then identifies the maximum-margin hyperplane in that space. Feature scaling is mandatory for SVM because the RBF kernel computes Euclidean distances — unscaled features with larger numerical ranges would dominate. The SVM provides an independent confirmation of the non-linearity finding through a mechanistically different approach from the Random Forest.

**Model 4 — Long Short-Term Memory Neural Network:** The first sentence gave a basic overview of what the LSTM model is. It explains that it is the only model that accounts for the temporal sequence. Other models treat each trading day as an independent observation with no memory of past days. It maintains a ‘rolling window’ of 10 trading days as input. Further on, it

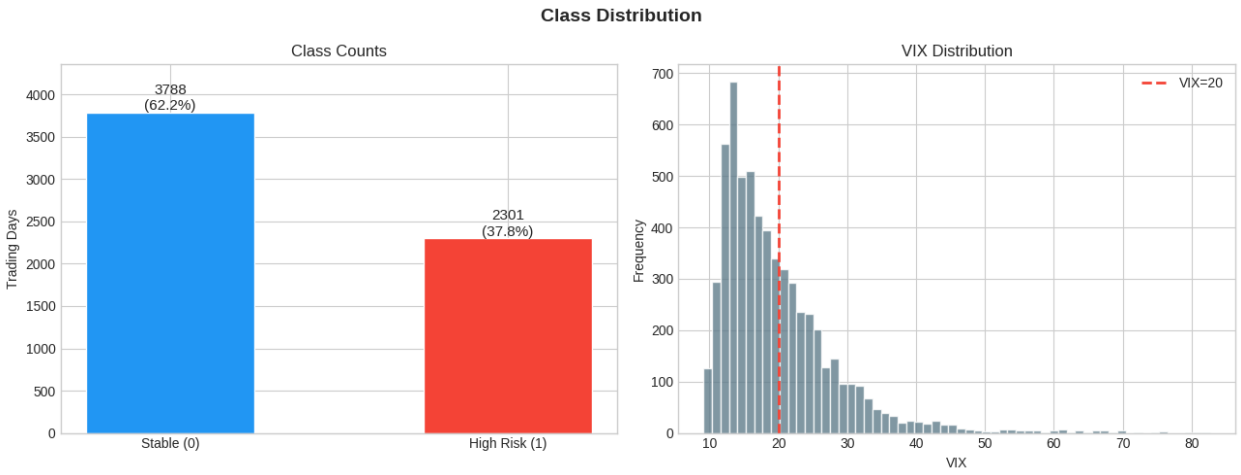
states that it uses mechanisms called gated memory cells to only remember important/required information as time passes. Hence, it can learn patterns over time like a gradually declining RSI over two weeks combined with a price falling steadily below MA. The architecture is a single LSTM layer of 32 units, followed by a dropout layer at 0.2 for regularization, a dense hidden layer of 16 units with ReLU activation, and a single-unit output layer with sigmoid activation. It is compiled with the Adam optimizer and binary cross-entropy loss and trained for a maximum of 30 epochs at a batch size of 32, with early stopping monitoring validation loss at a patience of 5 epochs and restoration of the best weights.

# 3. Results

## 3A. Results and Analysis

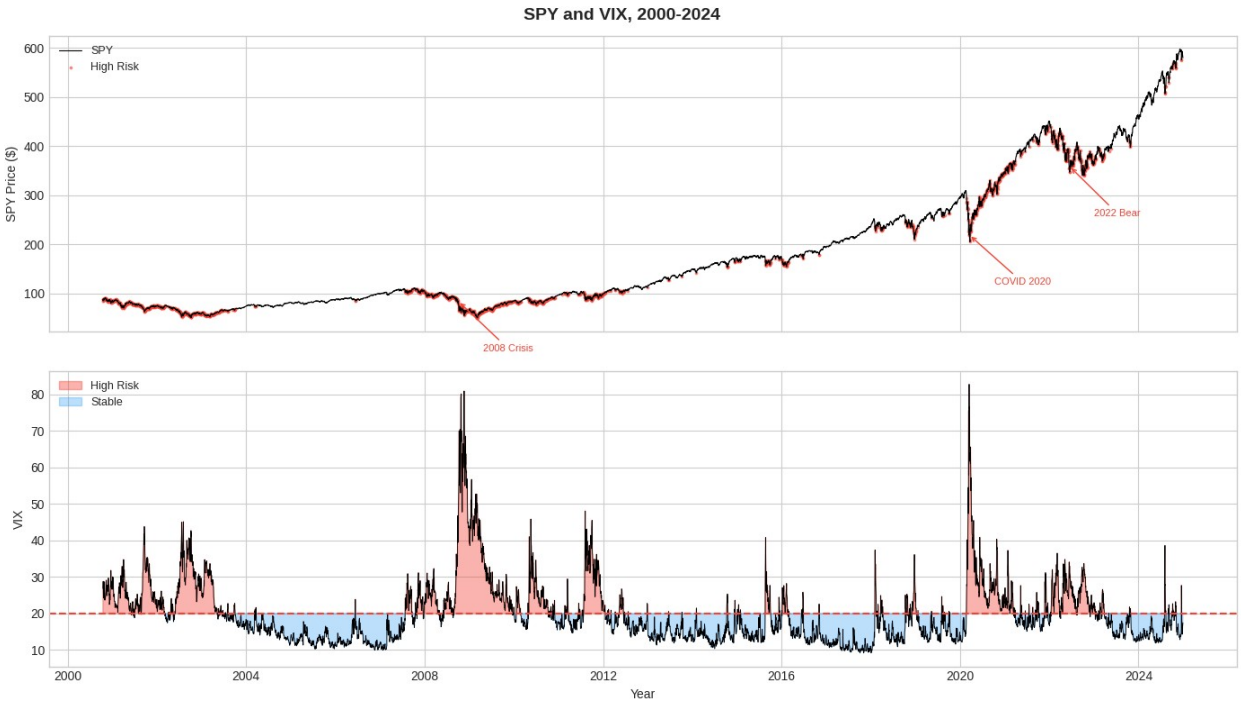
### Exploratory Data Analysis Results

**Figure 1 — Class Distribution and VIX Histogram**




The class distribution shows there is a big imbalance in the classes: [3788] trading days ([62.2]%) are labeled Stable and [2301] days ([37.8]%) are labeled High Risk across the full dataset. In the right panel, the VIX values have a right-skewed distribution with the majority of observations lower than 20 and the remaining of the observations go as high as 80 which include financial crises periods such as the 2008 crisis and 2020 COVID pandemic. Since the VIX values have a right-skewed distribution, it makes sense to make our threshold VIX > 20. If it was a symmetrical distribution, then splitting it in such a way would make no sense but the right-skew accounts for the separation of ordinary from fear intense distributions. A trivial classifier that predicted Stable for every single observation would still achieve 62.2% accuracy while providing no information whatsoever, which is why accuracy cannot be relied upon when measuring performance on this problem.

**Figure 2 — Historical Time-Series with Regime Labels**

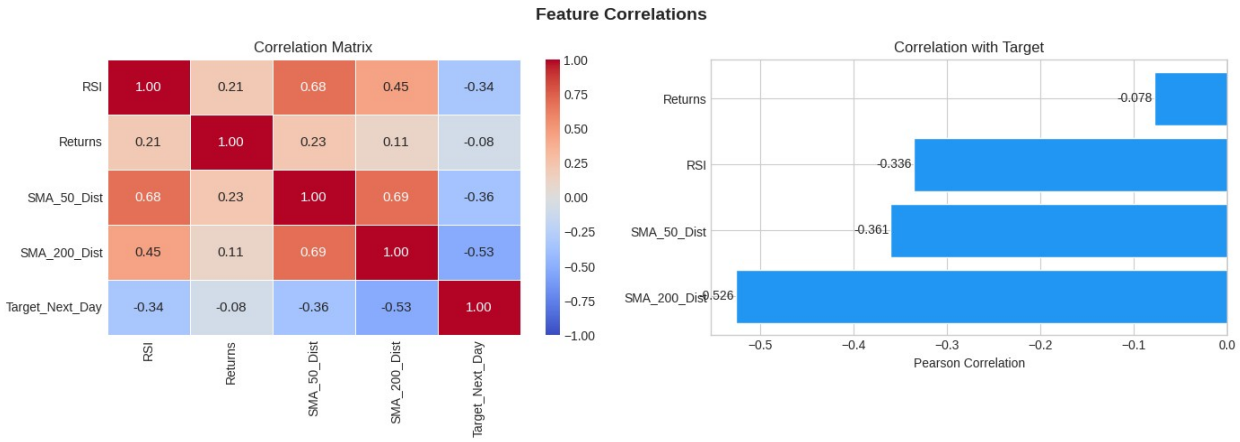



The time-series overlay confirms that the labeling logic is viable as it produces historically coherent and consistent results. On the left plot, the periods with tight clustering of high-risk labels correspond to major macroeconomic shocks such as the dot-com rise and collapse in 2000–2002, the financial crisis in 2008–2009 which constituted the longest sustained high-risk period (approximately 18 months) in the dataset, the 2011 European sovereign debt crisis, the 2015–2016 China-driven selloff, and finally the 2020 COVID-19 pandemic. In the VIX panel, corresponding to the bottom plot, there are sustained elevations during each of these periods showing agreement with our labeling.

But something critical to note is how crash regimes are not really spaced randomly over the time series and instead follow a pattern where they cluster around shocks and structural events and persist for long periods. This dependency or pattern in time is what drives the reasoning for

LSTM because if high-risk days are clustered i.e. dependent on the days before and after them the memory component of LSTM would be highly beneficial when classifying any given day compared to other models that would only consider the data for a single day.

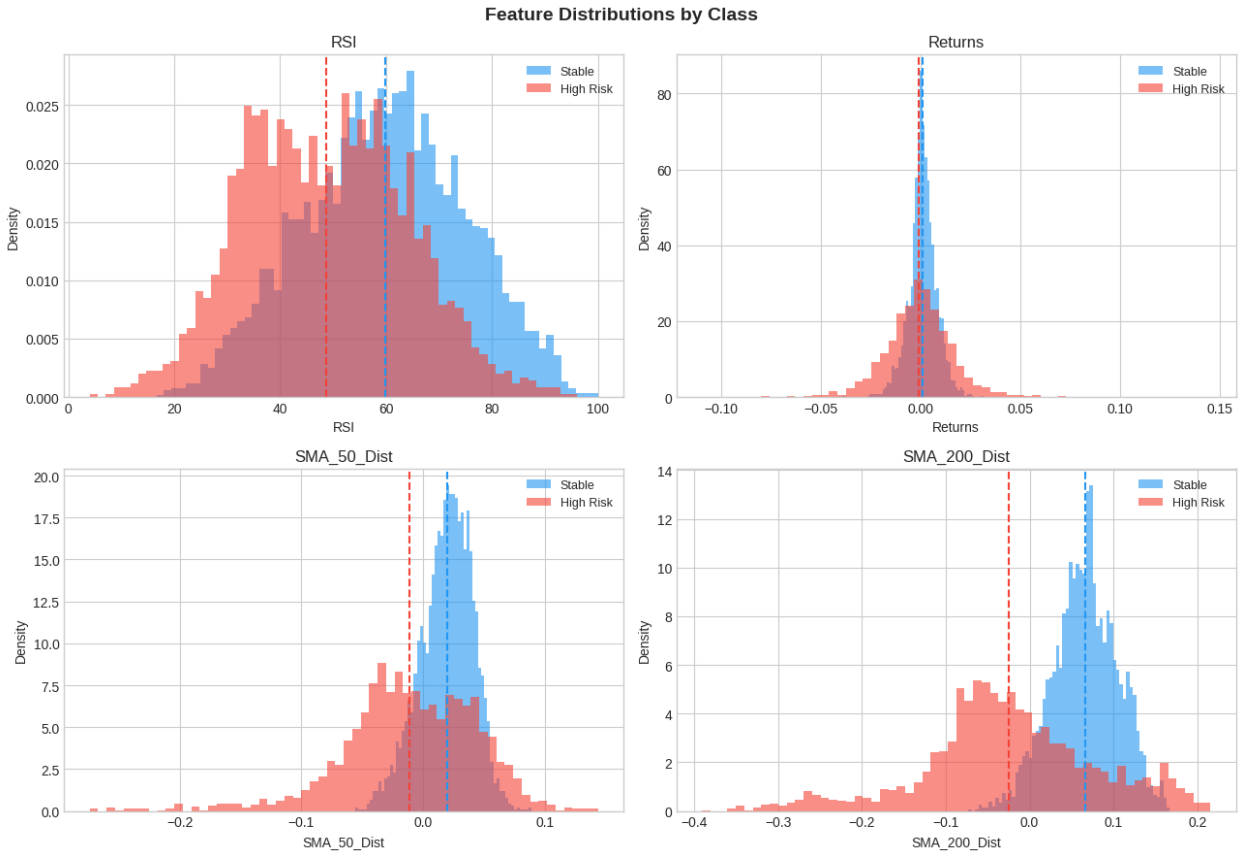
**Figure 3 — Feature Correlation Heatmap**




The correlation matrix reveals RSI: -0.3360, Returns: -0.0785, SMA_50_Dist: -0.3606, and SMA_200_Dist: -0.5263. All four correlations are negative and economically coherent: price below trend, weakening momentum, and negative returns each associate with elevated forward volatility. The magnitudes vary considerably. SMA_200_Dist at -0.5263 carries genuine linear signal, while Returns at -0.0785 carries almost none. What this analysis does not establish, however, is that the two classes are linearly separable. A moderate marginal correlation is entirely compatible with heavily overlapping class-conditional distributions, which is precisely what Figure 4 goes on to show: the classes differ in their means but overlap substantially through the bodies of their distributions. Discriminating power therefore has to come from the joint

interaction of all four features rather than from any single linear projection, which is the motivation for proceeding to non-linear models. There was also a moderate positive correlation between SMA_50_Dist and SMA_200_Dist at 0.69 which makes sense as both will show distance from a trend but I will include both in the model as they both have different weightages for the target as the 50-day trend shows some loss of strength while the 200-day trend shows a major momentum breakdown that clearly tells us that both factors are diverging and not related anymore and should be considered separately.

**Figure 4 — Feature Distributions by Regime**



The feature distributions do change a lot between classes, but there is a heavy overlap between the classes' distributions. The RSI tends to be lower on 'High Risk' days, with a mean of 48.61 instead of 59.8. The increase in variation is expected as RSI is an indicator based on previous stock prices and not independent, and as the other features lead to an increase in variation, the RSI should follow in some sense to portray the state of the market. The returns have a higher variance and slightly more negative mean as the sharp drops in prices would shake up the returns. The SMA distance features also drop and even though it rarely breaks both SMA distances (around 10% percent of the time), it occurs on many more occasions in the year that these crashes occurred, making it significant enough for the dates to be labeled as 'High Risk'. None of the indicators alone are enough for the labeling scheme, so we do require the entire combination to accurately label and predict if the market state is 'High Risk' or 'Stable'.

```
Logistic Regression
              precision    recall  f1-score   support

      Stable       0.69      0.95      0.80       653
   High Risk       0.91      0.54      0.68       604

    accuracy                           0.75      1257
   macro avg       0.80      0.74      0.74      1257
weighted avg       0.79      0.75      0.74      1257

Coefficients:
  SMA_200_Dist: -1.6452
  RSI: -0.4902
  SMA_50_Dist: +0.3002
  Returns: -0.0256
```

The Logistic Regression model showed a Recall of 0.54, Precision of 0.91, F1 Score of 0.68 and the Roc- AUC of 0.769. In the test set, there are a total of 604 High Risk Days. Out of these, Recall tells us it predicted 326 correctly and 278 incorrectly. These 278 False Negatives are actually all High-Risk Days and thus crucial misses. Regardless, upon investigating the coefficients :SMA_200_Dist = -1.6452, which indicates that the most important feature is the long-term trend breaking down since long-term trend accounts for the most negative crash days, as we saw in EDA. The second is RSI at -0.4902. All the coefficients' directions financially make sense.

F1, while bad, is okay since Precision > 0.60. However, there is a lack of important High Risk days as evidenced by worse Recall. 0.54 frankly is bad since it is less than our target of 60% and from our EDA, we can see that the target column feature correlation is near 0 so there is no way any hyperplane of L.R will separate them correctly thus providing ample base for explanations for poor Recall Score. Hence, while the L.R model is not good, it is not built incorrectly but rather built correctly, here the issue is the method itself.

Random Forest

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Stable | 0.71 | 0.93 | 0.81 | 653 |
| High Risk | 0.89 | 0.59 | 0.71 | 604 |
| accuracy | | | 0.77 | 1257 |
| macro avg | 0.80 | 0.76 | 0.76 | 1257 |
| weighted avg | 0.80 | 0.77 | 0.76 | 1257 |

The Random Forest got a Class 1 Recall of 0.59, a Precision of 0.89, an F1 Score of 0.71, and a ROC-AUC of 0.868. That shows almost a 0.10 jump in AUC when compared to Logistic Regression. Out of the 604 High Risk days, nearly 356 were correctly flagged, and 248 were missed. This means the Random Forest detected 30 extra true crashes compared to the Linear Model. The 5% increase in recall shows that market regime transitions involve non-linear, multi-feature interactions that can't be captured by a straight-line boundary. The ensemble can learn conditional rules, like the fact that low RSI combined with negative returns and negative SMA divergence signals danger—something that no hyperplane can approximate.

While the feature importance rankings showed SMA_200_Dist as the highest, this was consistent with its dominant coefficient in the Logistic Regression. This convergence across two mechanistically distinct models confirms long-term structural breakdown is the most reliable detectable precursor to High Risk regimes. RSI ranked second, confirming momentum deterioration as the next most informative signal. Despite the improvement, recall of 0.59 still falls just short of the 60% target, suggesting further complexity is warranted.

SVM

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Stable | 0.74 | 0.92 | 0.82 | 653 |
| High Risk | 0.88 | 0.66 | 0.75 | 604 |
| accuracy | | | 0.79 | 1257 |
| macro avg | 0.81 | 0.79 | 0.79 | 1257 |
| weighted avg | 0.81 | 0.79 | 0.79 | 1257 |

The SVM created some promising results, obtaining a Class 1 Recall of 0.66, Precision of 0.88, F1 Score of 0.75, and ROC-AUC of 0.870. For the 604 High Risk days, the SVM was able to get 399 correct but missed around 205, which was 43 less false negatives than the number of false negatives Random Forest achieved. The SVM is officially the first model to surpass the project's 60% recall target. Despite having a slightly higher precision which is in regards to how well the predicted values identified as high-risk actually were, we are prioritizing recall here instead of precision due to the aforementioned reasons. The RBF kernel performed as expected. It implicitly maps the four-dimensional feature space into an infinite-dimensional space, where a linear separating hyperplane corresponds to a flexible non-linear boundary in the original feature space, producing a tighter decision boundary than the Random Forest's axis-aligned threshold rules. It emphasized trying to avoid false negatives so that meant that it wanted to maximize the gap or the margin between the accurately predicted high-risk versus low-risk days by transforming the feature space. This overall allowed the model to get some improvement in

finding a good separation between classes compared to Random Forest and its threshold-based logic rules that it uses.

The ROC AUC value in this situation is very bad at fully explaining the situation, which is ironic since before I said it allows me to get a big picture but here, both models are very close in value towards it. The SVM is 0.870 while Random Forest is 0.868, but the SVM got better recall using the default 0.5 threshold. Clearly, this principle shows that operating thresholds matter greatly because a single metric like recall or ROC-AUC fails to give the giant picture. It simply fails.

LSTM

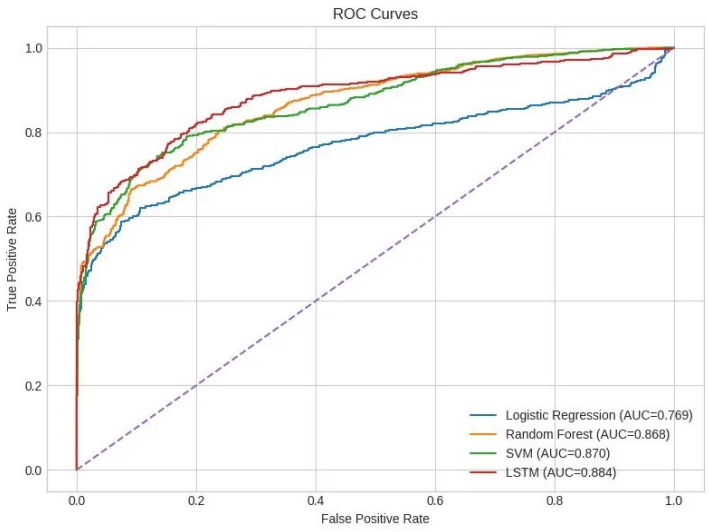
| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Stable | 0.77 | 0.88 | 0.82 | 643 |
| High Risk | 0.85 | 0.72 | 0.78 | 604 |
| accuracy | | | 0.80 | 1247 |
| macro avg | 0.81 | 0.80 | 0.80 | 1247 |
| weighted avg | 0.81 | 0.80 | 0.80 | 1247 |

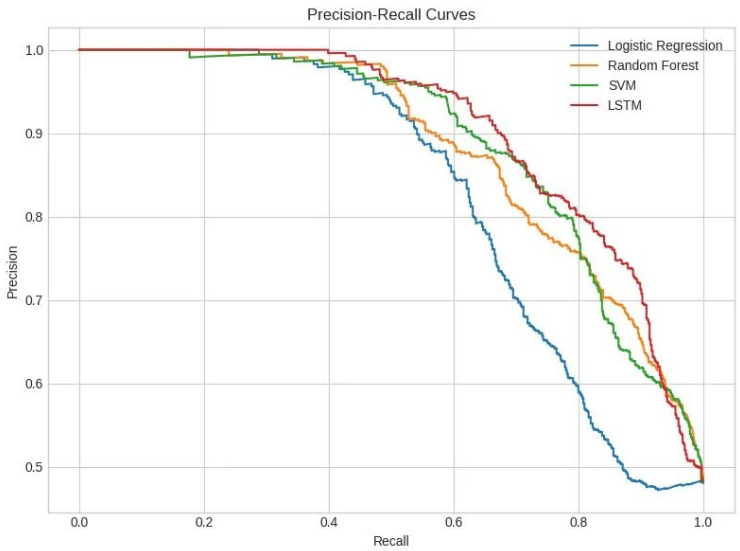
The results obtained from the LSTM model were very satisfactory as it performed the best among the four machine learning models in all metrics known for measuring model performance. The recall, precision, accuracy, F1 score, and ROC AUC were 0.72, 0.85, 0.80, 0.78, and 0.884 respectively indicating that the LSTM model is very effective at predicting market crashes when looking at the historical data.

The test set for my model contains 1247 values as the first 10 days of test data cannot form a full rolling window for LSTM. The analysis showed that out of 604 high-risk observations, the LSTM model predicted approximately 435 values correctly while around 169 were incorrectly predicted. The number of false negatives for this model is much lower in this model as compared to the other models as such as SVM and Random Forest. The LSTM produced 77 false positives against the SVM's approximately 54, meaning that roughly 23 additional false alarms bought 36 fewer missed spikes, reducing false negatives from 205 to 169. Under the cost asymmetry established earlier, this is a favorable trade: each false alarm costs approximately one day of foregone return, while each missed spike leaves the portfolio fully exposed to a drawdown.

Overall the result shows that the LSTM model is very effective at detecting the possible market crash and the cost asymmetry proves that to some extent. The increase in the recall value over other models shows that the sequential memory plays a significant role in making accurate predictions. The LSTM model looks at 10 days worth of data and then makes a prediction unlike the other models which makes it more effective. This means that LSTM is using the past values to look for any kind of market crash build-up path which happens over a 10-day period and then predict such as a major decline in RSI, the diverging SMA values, and negative returns. All of this combined leads to an accurate prediction for the LSTM showing the effectiveness of using sequential memory.

The training history also shows smooth convergence indicating the effectiveness of the dropout regularization which ensured that there was no overfitting in this model when it was predicting hence increasing the testing results.




The ROC curves confirm the model hierarchy visually. The Logistic Regression (blue) diverges noticeably from the non-linear models, confirming the inadequacy of the linear boundary beyond just the default threshold. The Random Forest (orange) and SVM (green) curves nearly overlap consistent with their near-identical AUC scores  while the LSTM (red) sits above all others across virtually the full plot. The Precision-Recall curves tell a complementary story: at recall levels between 0.6 and 0.8  the operationally relevant range for this project  the LSTM maintains higher precision than all other models, demonstrating it is not simply more aggressive in predicting High Risk, but genuinely more discriminating.

## 3B. Discussion

The progression of the four models paints a clear empirical picture. Starting with Logistic Regression's recall of 0.54, the recall consistently improves in a monotonic way across Random Forest, SVM, and LSTM with final recalls of 0.59, 0.66, and 0.72, respectively. Each improvement is directly linked to the unique capabilities of the algorithms aligning with the previous models' specific shortcomings: The linear model failed because no hyperplane separates the classes, while the ensemble model didn't fail but actually succeeded by taking into account combinations of multiple features. The SVM goes a step further by optimizing the boundary to maximize the margin in a high-dimensional space, with the LSTM going the extra mile to become the best by actually adding in the element of time, thus accessing a new dimension none of the others have access to. One caveat applies to all four results. The training period is 35.1% High Risk while the test period is 48.0%, because the 2020–2025 window contains the COVID-19 crash, the 2022 bear market, and the subsequent rate-hike cycle. Test metrics are therefore measured on a distribution meaningfully different from the one the models were fitted on. This is realistic, since deployment always faces an unknown future distribution, but it means these numbers should not be read as in-distribution estimates.

The trade-off between precision and recall across all these models shows why it is a logical one since all models that achieved a higher recall compensated for it by having a higher false positive rate. For our use case specifically, it is a logical trade-off since every true positive that is found

saves us from potentially huge market losses, while every false positive decreases our value only by roughly 0.04% expected return for that day. The 278 missed crashes from the Logistic Regression are a much bigger problem than having 77 false positives in the LSTM output and the results should reflect that.

It is important to explain the limitations of my approach, which I will do now. The features used are very limited since I only considered four technical indicators. A real production system would use far more factors such as interest rates, macroeconomic factors such as inflation or perhaps job growth or maybe even sentiment data. The VIX threshold is fixed at 20, which means that for it, any value greater than or equal to 20 is the same, completely ignoring all the different values greater than 20. The severe market crash due to COVID 19 occurs in the Test Set, which skews the performance results. More stabilized prediction sequences would occur in real financial markets. Finally, more parameters can be optimized for the LSTM which could increase accuracy.

## 3C. Conclusion and Recommendations

While working on this project, I learned that using machine learning to forecast market regimes is a practical and meaningful application of classification. I predicted each day's market by training my model on all previous days and avoided any future data leakage. During my

exploratory data analysis, I noted that while individual features did carry linear signal, the class-conditional distributions overlapped heavily, indicating that the decision boundary itself would not be linear. The first model, logistic regression, confirmed this by underperforming. Recall then improved monotonically across the remaining models: 0.54 for Logistic Regression, 0.59 for Random Forest, 0.66 for the RBF-kernel SVM, and 0.72 for the LSTM. Relative to the linear baseline, the Random Forest recovered 30 additional true positives, the SVM a further 43, and the LSTM a further 36, amounting to a 39% reduction in false negatives from end to end. The LSTM was therefore selected as the final model.

Achieving performance of 0.72 recall and 0.884 ROC-AUC, the LSTM caught around 72% of actual High Risk Days with a one-day offset. Precision was 0.85, meaning that 85% of the days flagged High Risk were correctly identified, against 77 false positives across the test set. This matters for a trading portfolio because failing to anticipate these days leaves positions fully exposed during the periods of largest drawdown. There's still room for improvement in the final model by adding more technical indicators and rate indicators like cross-asset signals and macroeconomic variables. Tuning the classification threshold away from the default 0.50 according to the portfolio's risk tolerance would allow the precision-recall balance to be set explicitly rather than inherited from a library default; the threshold sweep performed on the Random Forest quantifies this trade-off directly. Further, increasing the number of classes to Stable, Elevated, and Crisis would allow for more granular holding adjustments on each day for a more favorable result, potentially leading to less sensitive thresholds.

# 4. References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Wilder, J. W. (1978). *New concepts in technical trading systems*. Trend Research.

Yahoo Finance. (n.d.). *yfinance* [Software library]. PyPI. https://pypi.org/project/yfinance/